

OL Pro: Technical Documentation & Developer Guide

1. Project Overview

OL Pro is a server-side rendered (SSR) web utility designed to bridge the gap between Overleaf's dynamic project collaboration environment and the need for static, stable assets. It converts public Overleaf project share links into permanent, direct PDF download URLs. This tool is specifically optimized for professionals and students looking to embed or host their Overleaf-compiled documents (such as resumes or academic portfolios) without manual re-compilation or link breakage.

2. Architecture & Data Flow

The application utilizes an **API-proxy architecture** built on Next.js. The data flow follows a strictly server-side process to circumvent CORS restrictions and protect client-side environment configurations:

1. **Request Initiation:** The user submits a public Overleaf share link to the Frontend UI.
2. **Proxy Routing:** The request is routed to the server-side `app/api/overleaf/[token]/route.js`.
3. **Compilation Lifecycle:** The backend initiates a headless interaction with the Overleaf platform, acquiring necessary CSRF tokens and cookies.
4. **Integration & Extraction:** The backend triggers the compilation and parses the resulting internal payload to extract the stable PDF download URL.
5. **Response:** The server returns the PDF link, which is either rendered via the `PDF Viewer` component or provided as a download link to the user.

3. Core Components

- **Frontend UI:** Built with React and Tailwind CSS, this interface serves as the entry point (`app/page.tsx`). It handles user input validation and displays the final proxied PDF links.

- **Overleaf API Handler:** Contained within the `lib` and `api` modules, this component manages the compilation lifecycle. It is responsible for session authentication with Overleaf and orchestrating the request chain.
- **In-Memory Cache:** A lightweight storage layer using `NodeCache` to store session metadata and PDF locations.
- **PDF Viewer:** A specialized route (`app/view/[token]/page.js`) that hosts an iframe-based view for seamless document previewing.

4. Setup & Configuration

Prerequisites

- Node.js (LTS version recommended)
- npm or yarn

Installation

1. Clone the repository.
2. Install dependencies: `bash npm install`
3. Configure environment variables in a `.env.local` file (as required by the API integration).
4. Start the development server: `bash npm run dev` The application will be available at `http://localhost:3000`.

5. API Reference

`/api/overleaf/[token]`

This endpoint acts as the primary proxy for the Overleaf compilation process.

- **Method:** `GET`
- **Parameters:** `token` (The unique string extracted from an Overleaf share URL).

Logic:

- Checks the `In-Memory Cache` for an existing PDF URL associated with the `token`.

- If cache miss: Executes the authentication handshake with Overleaf, triggers the compilation, and caches the resulting URL.
- Returns a JSON object containing the status and the permanent PDF link.

6. Caching Strategy

The application employs `NodeCache` to optimize performance and adhere to rate-limiting best practices. * **Stored Data:** The cache holds CSRF tokens, project-specific cookies, and the final resolved PDF URLs. * **Lifecycle:** Data is set with an expiration duration defined within the `lib` module. * **Invalidation:** The cache is self-managed; however, users can force a refresh by re-submitting the request if the specific implementation allows for cache bypass parameters.

7. Troubleshooting & Limitations

- **API Changes:** Because the tool proxies a third-party platform's internal workflow, changes to Overleaf's CSRF or session management headers may require updates to the `Overleaf API Handler`.
- **Rate Limiting:** Excessive requests to the Overleaf domain via the proxy may trigger rate limiting on their platform. Monitor server logs for `429 Too Many Requests` status codes.
- **Compilation Errors:** If a project fails to compile within Overleaf (e.g., due to syntax errors in the LaTeX source), the proxy will return an error status. Ensure the source project is in a "compilable" state before using the viewer.